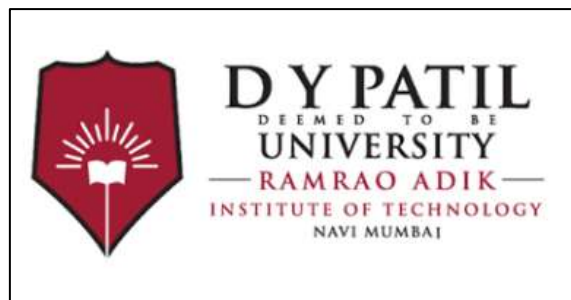


**Hardware-Based
USB Write Blocker
For
Digital Forensics**

Guide: Gautam Borkar



1. Introduction

In digital forensics, preserving the integrity of digital evidence is one of the most critical requirements. Any accidental or intentional modification of evidence can compromise an investigation and make the evidence legally inadmissible. When storage devices such as USB pen drives are connected to a system, operating systems may automatically write system files or update metadata, which can alter the original state of the evidence.

To prevent such modifications, write blockers are used. This project focuses on the design and implementation of a software-based USB write blocker using Linux and Python, which ensures that connected USB storage devices can only be accessed in read-only mode.

2. Problem Statement

When a USB storage device is connected to a computer, the operating system may automatically mount it with read-write permissions, update file access timestamps, or create hidden system files. These actions result in unintentional modification of digital evidence, violating forensic principles.

Although hardware write blockers exist, they are often expensive and inaccessible for students or small forensic laboratories. Therefore, a low-cost, software-based write-blocking solution is required.

3. Objective of the Project

The main objectives of this project are:

- To prevent any write operations on a USB storage device
- To allow safe examination of digital evidence
- To implement a low-cost, software-based alternative to hardware write blockers
- To understand and apply forensic best practices in evidence handling

4. What is a Write Blocker?

A write blocker is a forensic tool that allows investigators to read data from a storage device without allowing any modification to that data.

Types of Write Blockers:

1. **Hardware Write Blocker** – A physical device placed between the storage media and the computer
2. **Software Write Blocker** – A logical method implemented using operating system controls

5. Concept of Mounting in Linux

In Linux, storage devices cannot be accessed directly. They must be mounted to a directory in the file system.

- **Mounting:** Attaching a storage device to a directory so that its contents can be accessed
- **Read-Write (rw):** Allows reading and writing
- **Read-Only (ro):** Allows only reading and blocks all write operations

In digital forensics, devices must always be mounted in read-only mode to preserve evidence integrity.

6. Working Principle of the Software Write Blocker

The software write blocker works by:

1. Detecting the connected USB storage device
2. Preventing automatic read-write mounting
3. Mounting the USB device explicitly in read-only (ro) mode
4. Blocking any attempt to create, modify, or delete files
5. Allowing only safe read access for analysis

Tools and Technologies Used

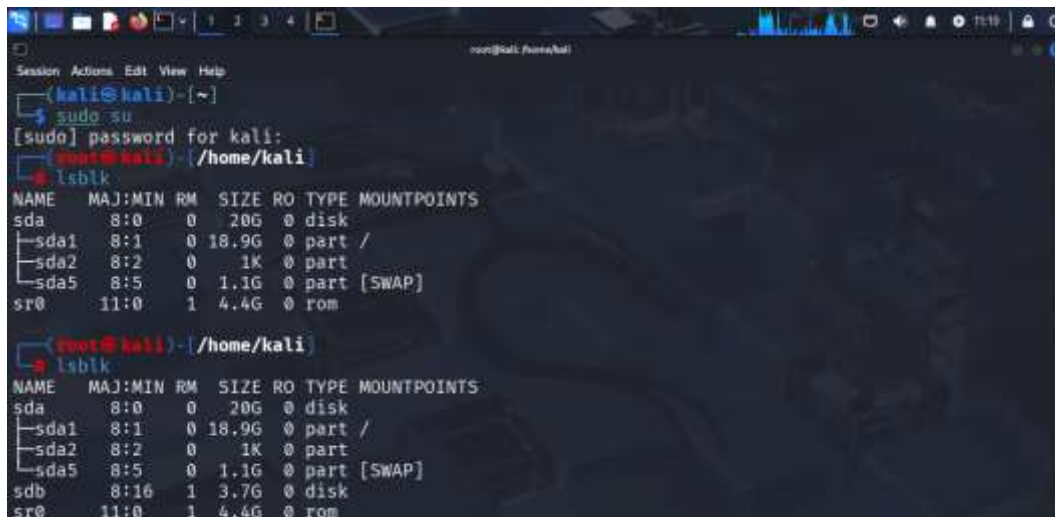
- **Operating System:** Kali Linux
- **Programming Language:** Python
- **System Utilities:**
 - `lsblk` – To detect storage devices
 - `mount` – To mount the device in read-only mode
 - `udisks2` – For device management
- **Python Libraries:**
 - `psutil` – For system and device monitoring

Implementation and Experimental Results

Step 1: Detection of USB Storage Device

Description:

The connected USB storage device was detected using the `lsblk` command. This command lists all block storage devices connected to the system and helps identify the evidence device without mounting it.



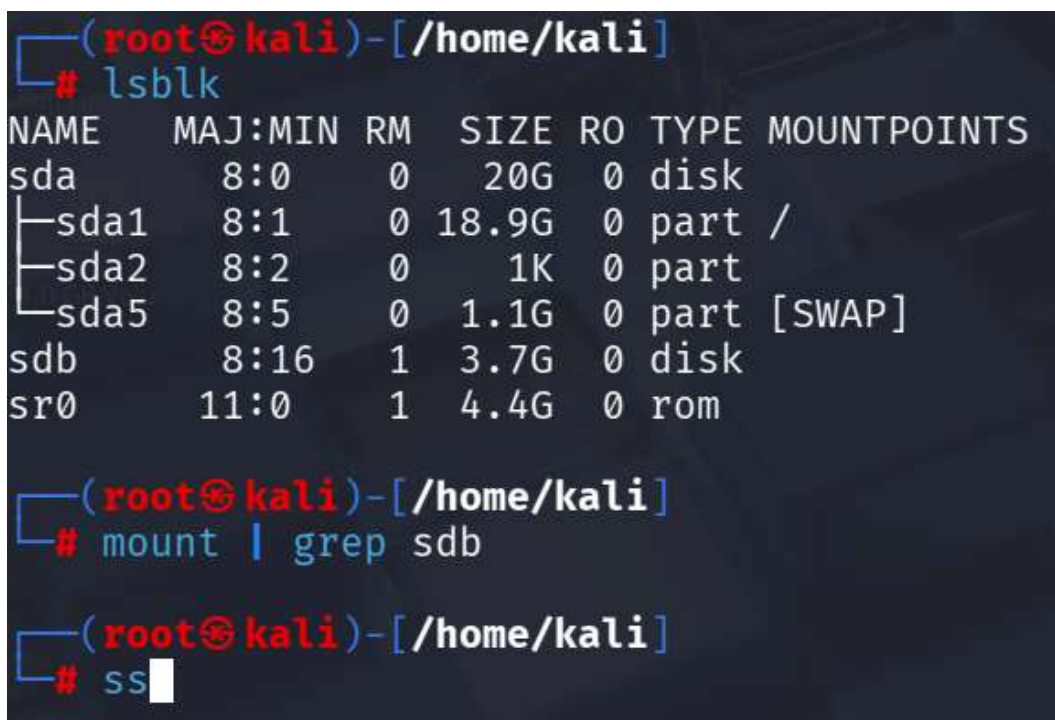
```
(kali@kali)~[~]
└─$ sudo su
[sudo] password for kali:
(root@kali)~[/home/kali]
└─# lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda   8:0    0  20G  0 disk
├─sda1 8:1    0 18.9G 0 part /
├─sda2 8:2    0   1K  0 part
└─sda5 8:5    0   1.1G 0 part [SWAP]
sr0   11:0   1   4.4G  0 rom

(root@kali)~[/home/kali]
└─# lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda   8:0    0  20G  0 disk
├─sda1 8:1    0 18.9G 0 part /
├─sda2 8:2    0   1K  0 part
└─sda5 8:5    0   1.1G 0 part [SWAP]
sdb   8:16   1   3.7G  0 disk
sr0   11:0   1   4.4G  0 rom
```

Step 2: Verification of Unmounted State

Description:

Before accessing the USB device, it was verified that the device was not automatically mounted to avoid any unintentional write operations.



```
(root@kali)~[/home/kali]
└─# lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda   8:0    0  20G  0 disk
├─sda1 8:1    0 18.9G 0 part /
├─sda2 8:2    0   1K  0 part
└─sda5 8:5    0   1.1G 0 part [SWAP]
sdb   8:16   1   3.7G  0 disk
sr0   11:0   1   4.4G  0 rom

(root@kali)~[/home/kali]
└─# mount | grep sdb

(root@kali)~[/home/kali]
└─# ss
```

Step 3: Creation of Secure Mount Directory

Description:

A dedicated directory was created to mount the USB device securely in read-only mode.

```
(root@kali)-[/home/kali]
# mount | grep sdb

(root@kali)-[/home/kali]
# mkdir -p /mnt/usb_ro
```

Step 4: Mounting USB Device in Read-Only Mode

Description:

The USB device was mounted in read-only mode to prevent any modification to the digital evidence

```
(root@kali)-[/home/kali]
# mkdir -p /mnt/usb_ro

(root@kali)-[/home/kali]
# mount -o ro /dev/sdb /mnt/usb_ro
```

Step 5: Verification of Read-Only Mount

Description:

The mount status was verified to confirm that the device was mounted with read-only access.

```
(root@kali)-[/home/kali]
# mount | grep sdb
/dev/sdb on /mnt/usb_ro type vfat (ro,relatime,fmask=0022,dmask=0022,codepage=437,ioccharset=asci
i,shortname=mixed,utf8,errors=remount-ro)

(root@kali)-[/home/kali]
#
```

Step 6: Read Access Test

Description:

The contents of the USB device were accessed to verify that read operations are allowed.

```
(root@kali)-[/home/kali]
# ls /mnt/usb_ro
Syllabus_Sem-VI.pdf  'System Volume Information'

(root@kali)-[/home/kali]
#
```

Step 7: Write Block Verification Test

Description:

A write operation was attempted to verify that write access was blocked.

Command Used:

```
(root@kali)-[/home/kali]
# touch /mnt/usb_ro/test.txt
touch: cannot touch '/mnt/usb_ro/test.txt': Read-only file system

(root@kali)-[/home/kali]
#
```

Conclusion

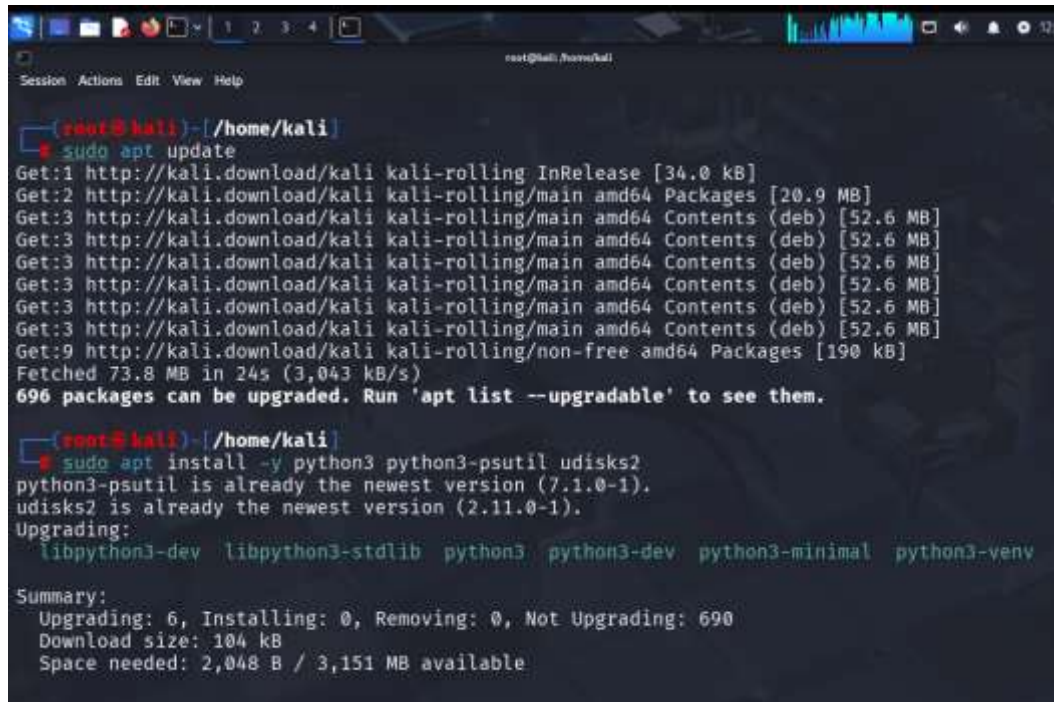
This project successfully implements a software-based USB write blocker using Linux to ensure safe handling of digital evidence. The USB device was mounted in read-only mode, and any attempt to perform write operations resulted in a *“Read-only file system”* warning. This confirms that the system effectively prevented modification of the evidence while allowing secure read access. The results demonstrate that a software-based write blocker can help maintain evidence integrity during forensic analysis.

Title: Automated Software-Based USB Write Blocker Implementation (Python)

Step 1: Install Required Packages

Objective:

To install the necessary system packages required for USB device detection and disk management.



```
(root@kali)-[/home/kali]
└─$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.6 MB]
Get:9 http://kali.download/kali kali-rolling/non-free amd64 Packages [190 kB]
Fetched 73.8 MB in 24s (3,043 kB/s)
696 packages can be upgraded. Run 'apt list --upgradable' to see them.

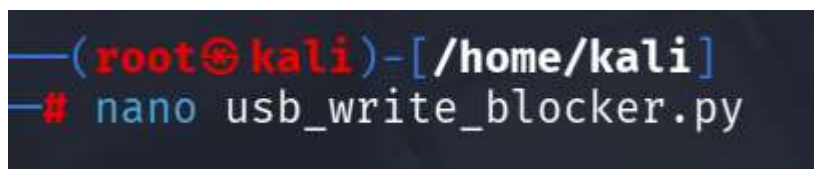
(root@kali)-[/home/kali]
└─$ sudo apt install -y python3 python3-psutil udisks2
python3-psutil is already the newest version (7.1.0-1).
udisks2 is already the newest version (2.11.0-1).
Upgrading:
libpython3-dev libpython3-stdlib python3 python3-dev python3-minimal python3-venv

Summary:
Upgrading: 6, Installing: 0, Removing: 0, Not Upgrading: 690
Download size: 104 kB
Space needed: 2,048 B / 3,151 MB available
```

Step 2: Create the Python Script

Objective:

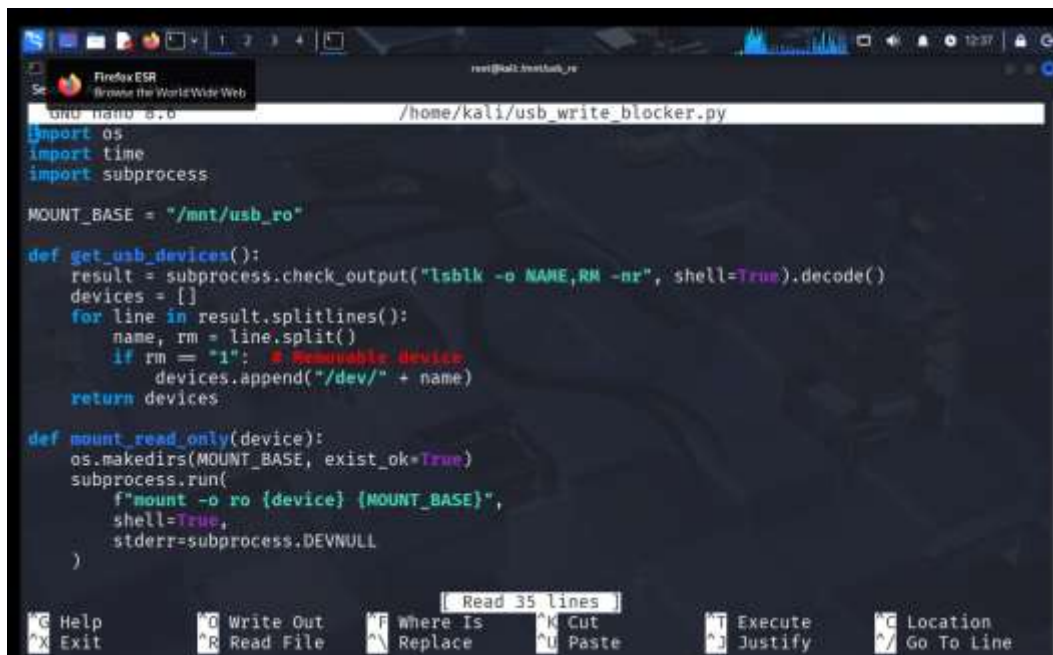
To create a Python script that will automatically detect USB devices and mount them in read-only mode.



```
(root@kali)-[/home/kali]
└─$ nano usb_write_blocker.py
```




```
GNU nano 8.6 usb_write_blocker.py
```



```
import os
import time
import subprocess

MOUNT_BASE = "/mnt/usb_ro"

def get_usb_devices():
    result = subprocess.check_output("lsblk -o NAME,RM -nr", shell=True).decode()
    devices = []
    for line in result.splitlines():
        name, rm = line.split()
        if rm == "1": # Removable Device
            devices.append("/dev/" + name)
    return devices

def mount_read_only(device):
    os.makedirs(MOUNT_BASE, exist_ok=True)
    subprocess.run(
        f"mount -o ro {device} {MOUNT_BASE}",
        shell=True,
        stderr=subprocess.DEVNULL
    )
```

The Python script continuously monitors USB connection events using system-level device monitoring and automatically mounts detected USB storage devices in read-only mode, ensuring that no write operation can modify the digital evidence.

Step 4: Execute the Script

Objective:

To start the automated USB monitoring and write-blocking service.



```
(root@kali)-[/home/kali]
└─$ sudo python3 usb_write_blocker.py
USB Write Blocker Service Started
/dev/sdb mounted in READ-ONLY mode
```


Step 5: Insert USB Storage Device

Objective:

To verify automatic detection and read-only mounting of a USB device.

```
(root@kali)-[/home/kali]
└─$ sudo python3 usb_write_blocker.py
USB Write Blocker Service Started
/dev/sr0 mounted in READ-ONLY mode
/dev/sdc mounted in READ-ONLY mode
```

Step 6: Verify Write Protection

Objective:

To confirm that write operations are blocked.

```
(root@kali)-[/home/kali]
└─$ cd /mnt/usb_ro

(root@kali)-[/mnt/usb_ro]
└─$ touch test.txt
touch: cannot touch 'test.txt': Read-only file system

(root@kali)-[/mnt/usb_ro]
└─$
```

Step 7: Verify Read Operation

After mounting the USB storage device in read-only mode, a read operation was performed to verify that data could be accessed without allowing any modification to the evidence.

```
Session Actions Edit View Help
root@kali: /mnt/usb_ro

(root@kali)-[/mnt/usb_ro]
└─$ cat /mnt/usb_ro/Syllabus_Sem-VI.pdf

%****1.6
earized 1/L 1471508/O 11723/E 122286/W 65/T 1470211/H [ 501 687]>>
<</DecodeParms<</Columns 5/Predictor 12>>/Filter/FlateDecode/ID[<A4E2CA612A060743984C74D860FEA9B
A><9620E1E17DA4424AAC2B01887CA36EB4>]/Index[11721 26]/Info 11720 0 R/Length 81/Prev 1470212/Root
11722 0 R/Size 11747/Type/XRef/W[1 3 1]>>stream
h*bbd`b`"e*d0**g**I0*"eJ*jR*$**b*!DN***Uk*||t#**
                                *d***h0*

startxref
0
%%EOF

<</C 1121/Filter/FlateDecode/I 1143/Length 593/S 1053>>stream
h*b`-r*c
    +
    cb*  p!T`+y**U+I**+
                                +g*****2+5*`x|aoA**+|.\\**B\\w***D**
                                ***+i{v5***      ^*Dh%pLm*x*/Y**
**~@_+bu<+pe**#"(A|g?+F+Fb@*3Z
                                **\\.M***'du*|****u+a**V****r***3g*Z*j****;g@***i*
                                *6%**N!*_*tL**kY
***S***(u*P****,*jxa**K'XV)X*3*!U*,+H**M**N**pM*:~11**划35EA*標.+3^***$
*Y
```

Conclusion

The implemented software-based USB write blocker successfully prevents any write operations while allowing read access to the evidence device. This ensures that digital evidence remains unmodified and fully accessible for analysis.

By automating the detection and read-only mounting of USB devices using Python, the system reduces human error and streamlines the forensic workflow. Overall, the project demonstrates a reliable, cost-effective, and professional approach to maintaining evidence integrity in digital investigations.